

Improving Robustness of Mathematical Reasoning in LLMs with Richer Chain-of-Thought Prompting

Tianyi Zheng

t4zheng@ucsd.edu

1 Introduction

Recent work into the reasoning capabilities of LLMs has claimed that LLMs are not yet truly capable of formal reasoning (Jiang et al., 2024; Shi et al., 2023). This poses a particular problem in the subdomain of mathematical reasoning, where it has been demonstrated that the capabilities of LLMs to reason through mathematical problems are extremely fragile (Mirzadeh et al., 2024). Most interestingly, these capabilities plummet most drastically when irrelevant information is added to training questions, which should not be expected if LLMs are truly performing formal mathematical reasoning. The goal of my project was to build upon the work of Mirzadeh et al. (2024) and assess whether richer chain-of-thought (CoT) prompting could make these capabilities more robust in the presence of irrelevant information.

Below is a list of the broad phases of my project:

- Collect and sample GSM8K dataset: DONE
- Annotate samples to make templates: DONE
- Generate synthetic datasets: DONE
- Evaluate models on synthetic datasets: DONE

2 Related work

My project builds upon the work of Mirzadeh et al. (2024), which investigates the fragility of mathematical reasoning in LLMs. The paper introduces GSM-Symbolic, a new benchmark based on the Grade School Math 8K (GSM8K) dataset (Cobbe et al., 2021), which is itself a popular benchmark for evaluating the mathematical reasoning capabilities of LLMs. GSM-Symbolic builds upon GSM8K by adding variation into the dataset in order to assess the robustness of LLM mathematical reasoning under such conditions. Evaluations with

GSM-Symbolic are performed with CoT prompting, which has been demonstrated to be effective in improving the performance of LLMs on complex reasoning tasks (Wei et al., 2023).

Most pertinently to my project, GSM-Symbolic includes GSM-NoOp, a dataset that adds “seemingly relevant but ultimately irrelevant information” to problems from GSM8K (Mirzadeh et al., 2024). Ultimately, the performance of numerous state-of-the-art LLMs on GSM8K problems plummets when evaluated with GSM-NoOp, suggesting that LLMs do not yet perform true formal mathematical reasoning. The authors hypothesize that LLMs rely on pattern matching and not true reasoning, and this corroborates existing literature on this topic (Jiang et al., 2024; Shi et al., 2023).

3 Your dataset

Since my project follows Mirzadeh et al. (2024), my dataset is also derived from GSM8K. My goal was first to replicate the results for the GSM-NoOp dataset as an initial baseline and then to expand upon those results with richer CoT prompting. However, the GSM-Symbolic dataset is unfortunately not publicly available, and neither is any code used to produce GSM-Symbolic.

To make my evaluation setup similar to that of Mirzadeh et al. (2024), so that I may establish a reasonable baseline for my experiments, I had to develop my own (much smaller) datasets using GSM8K based on the experimental setup detailed by Mirzadeh et al. (2024). I developed two separate datasets: `synthetic_data` for regular synthetic questions and `synthetic_data_no_op` for NoOp synthetic questions. Each dataset consists of 50 batches, with each batch containing 20 questions.

3.1 Data preprocessing

I did not need to perform any preprocessing of the GSM8K dataset to develop my evaluation datasets.

GSM8K	GSM Symbolic Template
Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?	{name}'s {bird}s lay {x} eggs per day. She eats {y} for breakfast every morning and bakes muffins for her friends every day with {z}. She sells the remainder at the farmers' market daily for \${w} per fresh {bird} egg. How much in dollars does she make every day at the farmers' market?
Janet sells $16 - 3 - 4 = <<16-3-4=9>>9$ duck eggs a day. She makes $9 * 2 = <<9*2=18>>18$ every day at the farmer's market. #### 18	<pre>#variables: - name = sample(names) - bird = sample(["duck", "chicken", "hen", "quail"]) - x = range(2, 100) - y = range(1, 100) - z = range(1, 100) - w = range(1, 100) - ans = range(0, 10000) #conditions: - x >= y + z - ans == w * (x - y - z)</pre> {name} sells {x} - {y} - {z} = {x - y - z} {bird} eggs a day. She makes {x - y - z} * {w} = \${ans} every day at the farmer's market. #### {ans}

Figure 1: Comparison of my manually created GSM symbolic templates to the original problems from GSM8K. Annotating the GSM8K problems into symbolic templates allows for the automated generation of a much larger dataset from a relatively small subset of the GSM8K dataset. Unlike [Mirzadeh et al. \(2024\)](#), my templates drop the calculator annotations, delimited by `<<...>>`, that are present in the original GSM8K data. I do not need these annotations, since my project does not concern training or fine-tuning models on GSM8K, as [Cobbe et al. \(2021\)](#) do.

3.2 Data annotation

I started with questions from the GSM8K test set and manually annotated them into symbolic templates (see Figure 1). For each template, I replaced all numbers, names, and major concepts in a GSM8K question with placeholder variable names. These variables had to be constrained by various logical conditions to ensure that only valid synthetic questions (mathematically sound and having non-negative integers as answers) could be generated, as long as the conditions were satisfied.

For my No-Op dataset, I inserted clauses with irrelevant information into my existing templates to create a separate set of No-Op symbolic templates (see Figure 2). I also annotated the answer portion of each No-Op template with an exposition of the relevance/irrelevance of every number in the question. These explanations are included during 8-shot CoT prompting to serve as richer prompts than those used by [Mirzadeh et al. \(2024\)](#).

I found the annotation process to be very laborious and time-consuming because of the manual nature of the task and because of the attention to detail required to avoid mistakes. I ultimately spent 2 weeks of on-and-off time solely on annotation. As

such, I only annotated 20 templates, corresponding to the first 20 questions of the GSM8K test set.

With those templates, I was able to generate synthetic questions automatically using OpenAI API calls to GPT-4o, which was able to populate the templates according to my predetermined conditions. I needed to rely on an LLM to generate the synthetic questions because, given the extremely limited time that I have for this project, any method that was not automatic would have been infeasible.

4 Baselines

Given that this project builds upon the work of [Mirzadeh et al. \(2024\)](#), my baselines will naturally be the GSM-Symbolic experiments from their paper. More specifically, my two baselines are the regular GSM-Symbolic experiment and the GSM-NoOp experiment. I ran the same experiments as they do on my chosen models, but on a much smaller scale simply due to my limited time and training data. As such, I expected the performance on my benchmarks to be lower than those of [Mirzadeh et al. \(2024\)](#). However, since all my experiments will be on an equally small scale, my results still serve as a good baseline.

GSM No-Op Symbolic Template

{name}'s {bird}s lay {x} eggs per day. She eats {y} for breakfast every morning and bakes muffins for her friends every day with {z}. She sells the remainder at the farmers' market daily for \${w} per fresh {bird} egg. **The amount of money she makes now is {v} times what she made per day last week.** How much in dollars does she make every day at the farmers' market?

```
#variables:
- name = sample(names)
- bird = sample(["duck", "chicken", "hen", "quail"])
- x = range(2, 100)
- y = range(1, 100)
- z = range(1, 100)
- w = range(1, 100)
- v = range(2, 10)
- ans = range(0, 10000)

#conditions:
- x >= y + z
- ans == w * (x - y - z)
```

We need to use the information of {x} eggs laid per day, {y} eggs eaten every morning, and {z} eggs used to bake muffins every day in order to determine how many eggs {name} sells per day.
We need to use the information of \${w} per egg to determine how much money {name} makes every day from selling her eggs.
We don't need to use the information that {name} makes {v} times what she made per day last week because we only care about determining how much she makes now.

{name} sells {x} - {y} - {z} = {x - y - z} {bird} eggs a day.
She makes {x - y - z} * {w} = \${ans} every day at the farmer's market.
{ans}

Figure 2: The GSM symbolic template from Figure 1 modified into a No-Op template. The **red text** is the added irrelevant information, and the **blue text** is the exposition that is used for richer CoT prompting.

5 Your approach

For this project, I chose to evaluate two models: OpenAI's GPT-4o and GPT-4o-mini. My choice of these models was due to their ease of accessibility through OpenAI's text completion API, and I was too limited by time and money to evaluate more models.

Each model was evaluated with 8-shot CoT prompting on my datasets `synthetic_data` and `synthetic_data_no_op` (see Figure 3 for the prompt format). More specifically, each model was evaluated twice using `synthetic_data_no_op`. One evaluation pass with the full No-Op questions (with exposition annotations) evaluated the models with a richer 8-shot CoT prompt and was the core of my experiment for this project. Another evaluation pass with reduced No-Op questions (without exposition annotations) determined my No-Op baseline.

The 8-shot prompting questions were created from the same templates as the datasets, with the 8 templates being sampled uniformly at random. A model's accuracy on each batch is the percentage of questions in which the ground-truth final answer was found in the model's output (excluding invalid questions, which are an issue that is addressed in Section 6).

5.1 Implementation

I have a working implementation of the approach described above with complete evaluation results.

The file `collect_gsm8k_data.py` saves 20 GSM8K questions to text files. I annotated these files to create my symbolic templates, as noted in Section 3.1. `generate_datasets.py` generates my two datasets by querying GPT-4o (see Figure 4 for the prompt format). `eval_models.py` evaluates my chosen models on the datasets.

5.2 Compute

I ran all code on my personal laptop, which has an AMD Ryzen™ 7 7840U CPU. However, my own compute resources are by and large irrelevant, as the main bottlenecks in my compute resources was the response time and cost of OpenAI API calls. During experimentation, dataset development, and evaluation, I used over 13 million tokens for GPT-4o and over 12 million tokens for GPT-4o-mini. In total, my use of the OpenAI API cost me \$47.58.

5.3 Runtime

The runtime of my code was extremely long, with almost all runtime spent on OpenAI API calls. To my memory, `generate_datasets.py` took over 3 hours to complete for my dataset size of 50

Evaluation Prompt Format

```
// System instruction
As an expert problem solver, solve step by step the following mathematical questions.

// User message, shot 1
Q: {question}
// Assistant message, shot 1
A: Let's think step by step. {solution}. The final answer is {final.answer}.

...

// User message, shot 8
Q: {question}
// Assistant message, shot 8
A: Let's think step by step. {solution}. The final answer is {final.answer}.

// User message, evaluation question
Q: {question}
```

Figure 3: The prompt format used for evaluations. This prompt was provided to the model via OpenAI’s text completion API using system, user, and assistant messages. The model is always given the same 8-shot questions in each evaluation.

Template Completion Prompt Format

```
// System instruction
You are a helpful assistant. The user will give you a question template with a question statement,
variable definitions, and conditions. The user will also give you an answer template with an answer
statement using the same variables as the question statement. Initialize values for the defined
variables, and ensure that the variable values satisfy the conditions. Populate both the question
statement and the answer statement with those variable values. Your numbers do not have to be
realistic or nice to work with, as long as the conditions are satisfied. Preserve all formatting,
including line breaks.

// User message
Question template:
'''
{question_template}
'''

Answer template:
'''
{answer_template}
'''
```

Figure 4: The prompt format used for generating synthetic questions from my annotated templates. This prompt was provided to the model via OpenAI’s text completion API using system and user messages.

batches, 20 templates, and 1 question per batch. Even worse, `eval_models.py` took over 12 hours to complete for just the two models of GPT-4o and GPT-4o-mini. Since I had to rerun my code many times during my annotation and implementation phases, I have most likely dedicated several days’ worth of compute time. The sheer amount of compute time required to run my experiments with my already-very-limited dataset sizes is one of the main reasons why I was unable to use larger datasets or evaluate more models.

5.4 Results

As shown in Figure 5, my evaluation results corroborate those of [Mirzadeh et al. \(2024\)](#): accuracy significantly drops from the `synthetic` dataset (regular synthetic questions) to the `no_op` dataset

(No-Op questions without exposition annotations). Thus, my baselines are broadly consistent with those of [Mirzadeh et al. \(2024\)](#). In addition to these results, it can be observed for both models that accuracy slightly increases from the `no_op` dataset to the `no_op_richer` dataset (No-Op questions with exposition annotations).

To verify that this slight increase in performance is in fact statistically significant, I ran one-tailed two-sample t -tests on the mean batch accuracy for `no_op` versus `no_op_richer` ($H_0 : \mu_1 = \mu_2$, $H_1 : \mu_1 < \mu_2$, $n = 50$). Refer to Figure 5 for the values of sample mean and standard deviation used for these statistical tests. For GPT-4o, the null hypothesis was rejected at the $\alpha = 0.01$ significance level ($T = -2.6766$, $p = 0.0044$). For GPT-4o-mini, the null hypothesis was also rejected

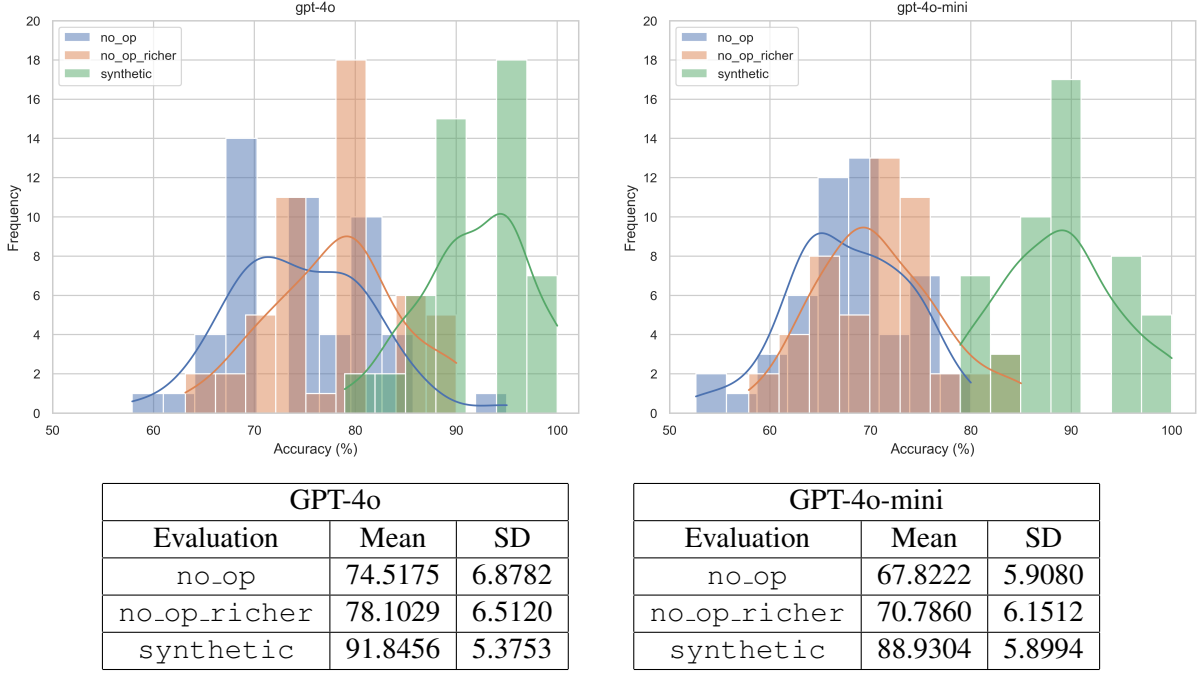


Figure 5: For both GPT-4o and GPT-4o-mini, accuracy significantly drops from the `synthetic` dataset to the `no_op` dataset, and variance also increases. However, accuracy slightly increases from `no_op` to `no_op_richer`.

at the $\alpha = 0.01$ significance level ($T = -2.4572$, $p = 0.0079$). Thus, I conclude that richer CoT prompting through the exposition of given information yields a statistically significant increase in mathematical reasoning performance for GPT-4o and GPT-4o-mini.

6 Error analysis

For my baseline evaluations, it was inevitable that the evaluated models would answer some questions incorrectly, since it is already well-established that LLM model reasoning is imperfect (Mirzadeh et al., 2024; Cobbe et al., 2021). However, I observed that it was common for GPT-4o-mini to answer a question incorrectly while GPT-4o answers that same question correctly. Examples like that in Figure 6 and the evaluation results shown in Figure 5 both demonstrate GPT-4o’s more robust mathematical reasoning capabilities in comparison to GPT-4o-mini, which was already established by Mirzadeh et al. (2024).

Instead, I identify my manual data annotation and my use of an LLM for dataset generation to be much more pertinent sources of error. That the symbolic templates were created by myself as an individual means that some mistakes in the templates, however minuscule, risks going unnoticed. While I encountered many mistakes in my templates during

the development process (and had to regenerate my datasets as a result), I did not notice any such mistakes in my final templates, but it is possible that I may have missed some subtle ones. Furthermore, my use of an LLM to generate my datasets using my templates results in datasets that will inevitably contain invalid questions. The LLM can and does occasionally generate a question that fails to satisfy all of the criteria specified in the template, even if the template in question is entirely correct. An example of this can be found in Figure 7. I try in my code to exclude these examples from evaluation, to some success, but I could not guarantee that all evaluation questions were entirely valid. Ironically, it is the imperfect, fragile mathematical reasoning capabilities of LLMs that impede my investigation of those same capabilities for this project.

7 Conclusion

This project was ultimately successful in that I was able to conduct all my experiments and evaluations as planned and in that I was able to obtain significant results despite my limited training data and time. Creating symbolic templates was by far the most difficult aspect of my project. Had I had access to the GSM-Symbolic dataset by Mirzadeh et al. (2024), I could have avoided days to weeks of work on this project.

Example No-Op Question with GPT-4o	Example No-Op Question with GPT-4o-mini
Alex decides to try flipping a house. He buys a house for \$500000 and then puts in \$40000 in repairs. This increased the value of the house by 10%. Alex also saved \$20000 by doing all the repair work himself. How much profit did he make? Let's break this down step by step. [Text omitted for brevity] The savings of \$20,000 from doing the work himself are not directly considered in the "profit" from the sale of the house, as they rather affect the cost side if the repairs would cost more otherwise impacting the total out-of-pocket expense. Thus, **the profit Alex made from selling the house is \$10,000.**	Alex decides to try flipping a house. He buys a house for \$500000 and then puts in \$40000 in repairs. This increased the value of the house by 10%. Alex also saved \$20000 by doing all the repair work himself. How much profit did he make? Let's break this down step by step. [Text omitted for brevity] - Alex saved \$20,000 by doing the repairs himself. This means his effective repair cost is: - Effective repair cost = \$40,000 - \$20,000 = \$20,000 - Now, the total investment, considering the saved labor cost, is: - Total Investment = \$500,000 + \$20,000 = \$520,000 - Profit = New value of the house - Total Investment - Profit = \$550,000 - \$520,000 = \$30,000 So, the total profit Alex made is **\$30,000**.

Figure 6: A No-Op question that GPT-4o answers correctly while GPT-4o-mini answers incorrectly. The red text is the added irrelevant information.

GSM8K
In a dance class of 80 students, 25% enrolled in tap dance, 50% of the remaining enrolled in ballroom dance, and the rest enrolled in interpretive dance. What percentage of the entire students enrolled in interpretive dance? There are $80 \times 25/100 = 20$ students who enrolled in tap dance. So, $80 - 20 = 60$ students are enrolled in either ballroom or interpretive dance. There are $60 \times 50/100 = 30$ students who enrolled in ballroom dance. Hence, $60 - 30 = 30$ students enrolled in hip-hop dance. This is $30/80 \times 100\% = 37.5\%$ of the entire students. #### 37.5

Figure 7: An invalid question generated by GPT-4o in my synthetic_data dataset. Although the template was correct and the problem is mathematically sound, questions should only have non-negative integer answers because answers are defined to be sampled from ranges of integers.

If I were to continue this project, the most obvious plan of action would be to scale up my experiments to use the entire GSM8K dataset. This would require me to convert the entire dataset to symbolic templates, so I would need to find some alternative, less manual method of annotation.

In any case, I am quite content with the results that I obtained from my model evaluations. I do not consider my conclusions to be particularly surprising, but I am content that I succeeded in obtaining meaningful results at all, given my circumstances.

8 Acknowledgements

As stated in Section 3.1, I used GPT-4o to generate my symbolic templates and my datasets. I did not use generative AI tools for any other part of this project, including the writing of this report.

References

- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems.
- Jiang, B., Xie, Y., Hao, Z., Wang, X., Mallick, T., Su, W. J., Taylor, C. J., and Roth, D. (2024). A peek into token bias: Large language models are not yet genuine reasoners.
- Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., and Farajtabar, M. (2024). Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models.
- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., and Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J., editors,

Proceedings of the 40th International Conference on Machine Learning, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023). Chain-of-thought prompting elicits reasoning in large language models.